Ho Chi Minh City University of Technology




Faculty of Computer Science and Engineering

Course: Computer Architecture Lab (CO2008)

# Assignment

## Checkers boardgame with MARS MIPS Assembly

Ho Chi Minh City, April 2025

# Contents

**1 Introduction** **2**

1.1 Forewords . . . 2

1.2 Basic rules of English checkers . . . 2

1.2.1 Setup . . . 2

1.2.2 Moving the pawns . . . 3

1.2.3 Capturing and chaining captures . . . 4

1.2.4 Promotion . . . 6

1.2.5 Winning the game . . . 7

1.3 Checkers notation . . . 8

**2 Requirements** **10**

2.1 Game state display . . . 10

2.2 Making a move . . . 10

**3 Extra tasks** **13**

3.1 Quality of Life . . . 13

3.2 Advanced rulesets . . . 14

**4 General** **15**

4.1 Basic Rules . . . 15

4.2 Expectations . . . 15

# 1 Introduction

## 1.1 Forewords

*Checkers* (also known as *draughts* in the UK) is perhaps one of the most common boardgame worldwide. Played on an $8 \times 8$ or $10 \times 10$ grid, the main objective of the game is to use one's own pieces exhaust the opponent of their pieces.

For the sake of simplicity, in this assignment, students are expected to implement the **English variant** of checkers using MARS MIPS Assembly. There are some other variants that students might attempt to implement for extra points.

## 1.2 Basic rules of English checkers

### 1.2.1 Setup

English checkers is played on a $8 \times 8$ grid between two opposing sides. Although there are 64 spaces in total, only 32 are ever used (as explained later).

One side will be in control of dark pieces (usually black), while the other will be in control of light pieces (usually red, sometimes white).

The pieces themselves are individually called "man" or "pawn". This document will opt to refer to them as the latter.

The pawns are initially laid on the board like so:

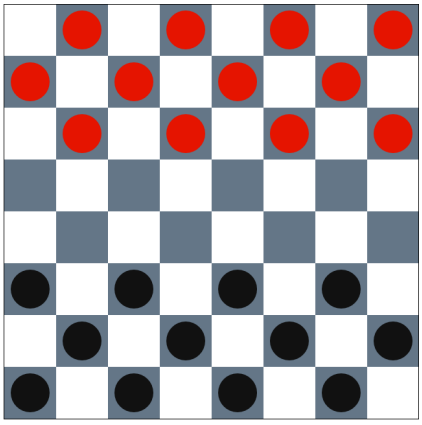


Figure 1: Initial position of pawns on the checker board

The player with dark pieces starts first. Then players alternate turns.

### 1.2.2 Moving the pawns

On each player's turn, they must advance one of their pawns one step diagonally forward to an empty space. As a result, all white spaces are inaccessible.

For example, here are the possible starting moves for black (the orange arrows):

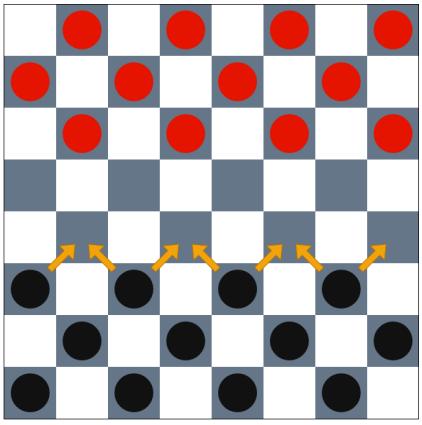


Figure 2: Starting moves for black.

### 1.2.3 Capturing and chaining captures

If the space one step forward is occupied by an enemy pawn and there an empty space immediately behind it, then a capture is possible.

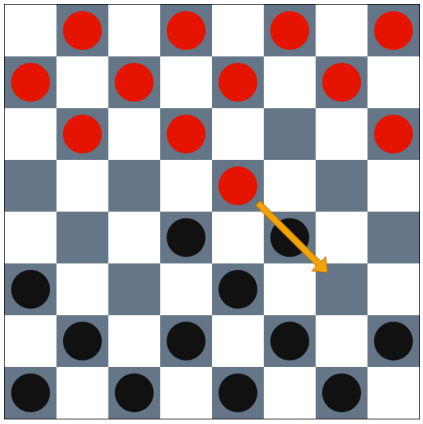


Figure 3: Red can capture by jumping over black and landing on the empty space directly behind it.

The capturing pawn will jump over the enemy pawn and land in the aforementioned empty space, while the captured enemy pawn will be removed from play.

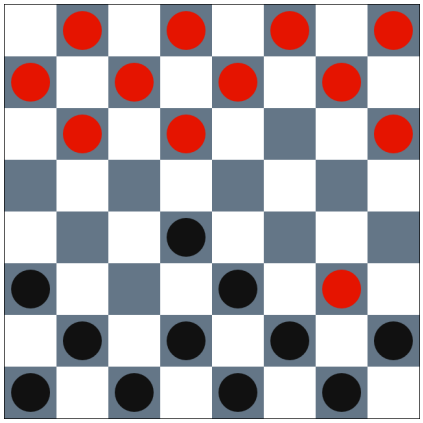


Figure 4: Game state after capture

If another capture becomes possible immediately after landing, then the player can perform that capture before their turn ends. This is called a chain capture. There is no limit to how many captures can happen within a chain.

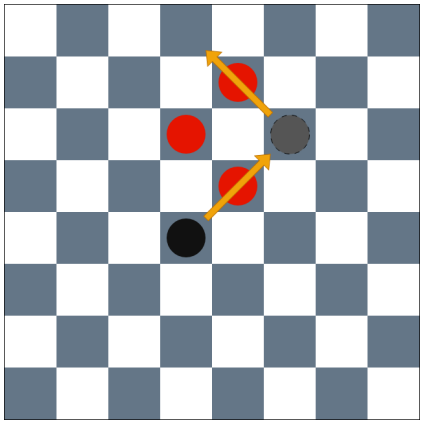


Figure 5: Black can make consecutive jumps and chain captures.

According to the English ruleset:

1. If an initial capture is possible, then it **MUST** be taken, even if it results in a disadvantage for the capturer.
2. If a chain capture following the initial capture is possible, then it **MUST** be taken

For the bare minimum implementation of this assignment, we shall ignore the first rule while following the second one.

### 1.2.4 Promotion

Once a pawn advance to the final row, it is promoted to a "king" piece.

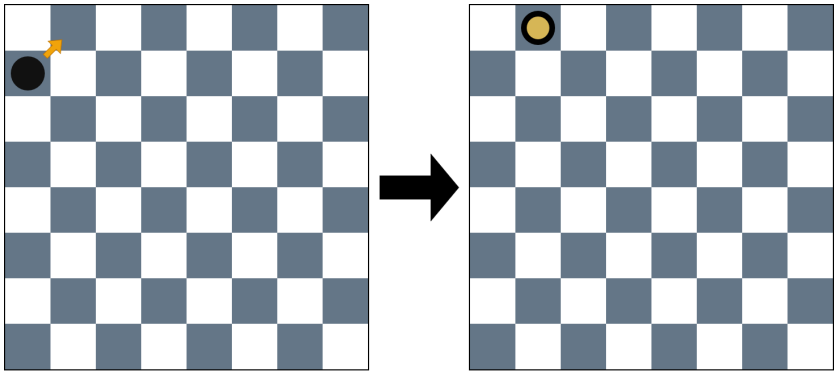


Figure 6: Promoting a black pawn to a black king.

In the English variant of checkers, unlike pawns, kings can move or capture both backwards and forwards. Their range is still the same as a pawn, however.

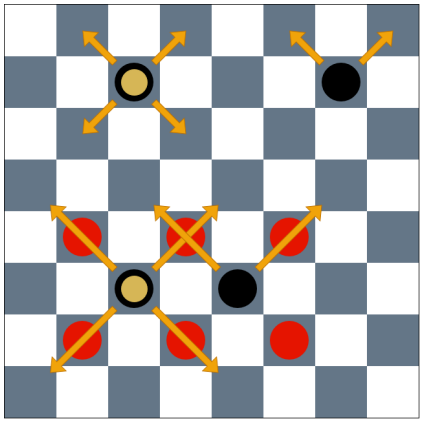


Figure 7: Movement ranges and capture ranges of black kings versus black pawns

If a pawn advances to the final row via a capture, it is instantly promoted to king and can immediately chain capture backwards.

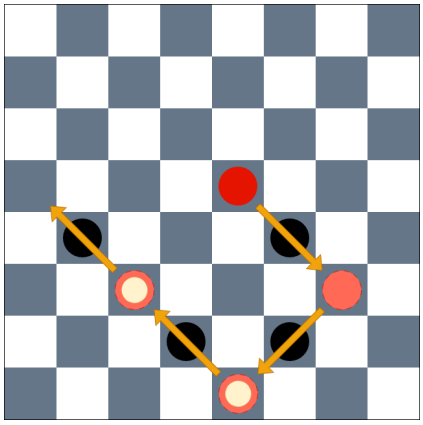


Figure 8: A newly promoted red king can instantly jump backwards to continue the capture chain.

### 1.2.5 Winning the game

A player wins the game by exhausting their opponent of any valid move. Usually this happens when all opposing pieces have been captured. Your implementation should at least check for this scenario, announce the winner, and then end the game.

A rarer case is when all of the opponent's pieces has been blocked off.

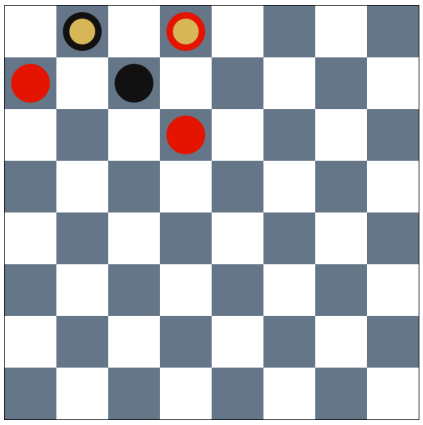


Figure 9: There is no valid move black can take, thus red wins the game.

## 1.3 Checkers notation

To aid with recording games, a standard notation system has been developed for checkers, much like with chess.

In this system, the accessible spaces on the grid is numbered like so:

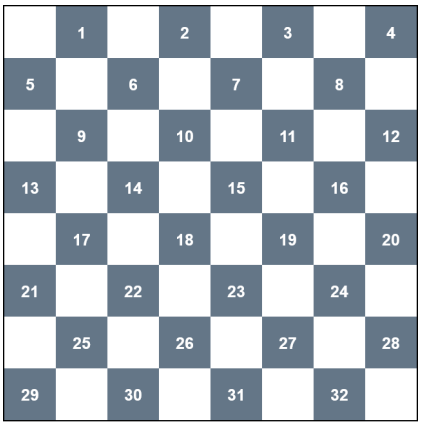



Figure 10: Grid number notation with the black side at the bottom

The bottom left square (29) will contain a black pawn at initial setup, while the top right square (4) will have a red pawn at startup.

For reference, here is the grid numbering chart aligned to the red side's view.

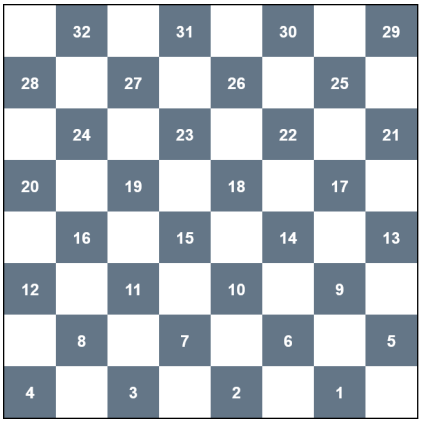



Figure 11: Grid number notation with the red side at the bottom

Each move is recorded as 2 numbers, first is the starting square and second is the landing square. A dash ('-') indicates normal movement, while an x or a cross ('×') indicates a capture jump.

For example:

- In 6, the move is recorded as: **5-1**
- In 8, the chain of capture is recorded as: **15x24 24x31 31x22 22x13**

# 2 Requirements

## 2.1 Game state display

Your program MUST be able to output to the MARS MIPS termninal the current state of the game board every turn. Below is a suggestion.

```
_ r _ r _ r _ r
r _ r _ r _ r _
_ r _ r _ r _ r
0 _ 0 _ 0 _ 0 _
_ 0 _ 0 _ 0 _ 0
b _ b _ b _ b _
_ b _ b _ b _ b
b _ b _ b _ b _
```

Where '_' represents inacessible spaces, 'r' and 'b' represent pawns of both sides, while 'R' and 'B' represent kings of both sides. '0' represents empty spaces.

## 2.2 Making a move

The game will prompt the current player to enter the starting and landing squares, then the program will check the validity of that move.

```
_ r _ r _ r _ r
r _ r _ r _ r _
_ r _ r _ r _ r
0 _ 0 _ 0 _ 0 _
_ 0 _ 0 _ 0 _ 0
b _ b _ b _ b _
_ b _ b _ b _ b
b _ b _ b _ b _
Black's turn.
Enter starting square: 21
Enter landing square: 17
_ r _ r _ r _ r
r _ r _ r _ r _
```

```
_ r _ r _ r _ r
0 _ 0 _ 0 _ 0 _
_ b _ 0 _ 0 _ 0
0 _ b _ b _ b _
_ b _ b _ b _ b
b _ b _ b _ b _
Red's turn.
Enter starting square:
```

Validity checks include:

- Does the starting square contain any piece?
- Does the piece in the starting square belong to the current player?
- Is it legal for that piece to move to the landing square? (In terms of direction and occupancy of the landing square)

If the checks fail then the player is prompted to try another combination.

```
Black's turn.
Enter starting square: 21
Enter landing square: 13
Not a valid move! Try again.
Enter starting square:
```

If the move is a capture and a chain is possible, prompt the user to enter the landing square to continue.

```
_ 0 _ 0 _ 0 _ 0
0 _ 0 _ 0 _ 0 _
_ r _ 0 _ 0 _ 0
0 _ B _ B _ 0 _
_ 0 _ 0 _ 0 _ 0
0 _ B _ B _ 0 _
_ 0 _ 0 _ 0 _ 0
0 _ 0 _ 0 _ 0 _
Red's turn.
Enter starting square: 9
Enter landing square: 18
```

```
Captured Black's piece.
_ 0 _ 0 _ 0 _ 0
0 _ 0 _ 0 _ 0 _
_ 0 _ 0 _ 0 _ 0
0 _ 0 _ B _ 0 _
_ 0 _ r _ 0 _ 0
0 _ B _ B _ 0 _
_ 0 _ 0 _ 0 _ 0
0 _ 0 _ 0 _ 0 _
Chaining possible.
Enter landing square:
```

# 3 Extra tasks

Your implementation of the checkers game following the requirements in Section 1 & 2 is considered the bare minimum product. The bare minimum product can only earn at most **9/10** marks (**8/10** marks for the Honors program).

To earn the remaining marks, students must implement some extra features. Below are some suggestions as to what you can do. Please note that the difficulty of the extra features and how well you've achieved them are the main factors in consideration for increasing your score.

## 3.1 Quality of Life

Certain Quality of Life (QoL) features can be added to the barebone game. In no particular order:

- Display the game board in a more beautiful way.
- Make the output of the current game state suit the current player. I. e. rotate the board so that it seems like red is the side moving up during the red side's turn.
- Going further on the previous idea, make the notation system suit the current player as well. For the player with red then their bottom left is also square 29.
- Make a different input system than the one presented. For example, let the player choose one square that contains their piece, then list all possible moves with that piece and tell the player to choose from the list instead. The same applies for chaining after a capture.
- Make more robust error messages for invalid inputs, tell the user what was wrong with it.

- Detect the winning condition of trapping the opponent with no valid move.
- Give the players the option to offer a draw. If both sides offer a draw, end the game in a draw.
- Allow the user to restart the game without ending the program.
- Anything you can come up with! So long as you give your rationale as to why it's a worthwhile QoL feature and explain how you implemented it.

## 3.2 Advanced rulesets

As mentioned in Section 1, the version of checkers in the minimum effort product is a trimmed down and simplified one.

For a bigger challenge, you may:

- Enforce capturing whenever it is possible.
- Expand the board to $10 \times 10$ with 15 or 20 starting pawns.
- Implement other variants of the game. For example, ones with flying kings and pawns that can move backwards from the start.
- Anything you think is harder!

# 4 General

## 4.1 Basic Rules

- Students must work in groups of 3 or fewer members.
- What is expected in your submission (report, source code, live demo, questioning, etc.) is up to the instructor and will be explained by said instructor through another channel.
- Students have the duty to keep their work secure and also to not copy off other group's work. Both parties's score will be impacted in the event plagiarism is detected.
- Usage of AI for coding is not forbidden, but understanding and designing should be the main goal.
- As only certain classes have this assignment, there will be no "harmony" questions in the final exam related to the work done here.

## 4.2 Expectations

Below are some, but not all, of the criteria students will be assessed on:

- Functionality of the software.
- Readability of the source code.
- Overall structure of the software.
- Understanding of the workings of MIPS Assembly.
- Skills in writing report and/or presenting the project's outcome.